Clean and Preprocess Data

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.ensemble import RandomForestRegressor
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
```

```
from google.colab import files
uploaded = files.upload()
```

Choose Files student-mat.csv

student-mat.csv(text/csv) - 56993 bytes, last modified: 3/25/2026 - 100% done

```
Saving student-mat.csv to student-mat.csv
```

```
# Load the data
df = pd.read_csv("student-mat.csv", sep=";")
df.head()
```

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	...	famrel	freetime	goout	Dalc	Walc	health
0	GP	F	18	U	GT3	A	4	4	at_home	teacher	...	4	3	4	1	1	3
1	GP	F	17	U	GT3	T	1	1	at_home	other	...	5	3	3	1	1	3
2	GP	F	15	U	LE3	T	1	1	at_home	other	...	4	3	2	2	3	3
3	GP	F	15	U	GT3	T	4	2	health	services	...	3	2	2	1	1	5
4	GP	F	16	U	GT3	T	3	3	other	other	...	4	3	2	1	2	5

5 rows × 33 columns

```
df.shape
```

```
(395, 33)
```

```
# Check missing values
print("Missing values per column:")
print(df.isnull().sum())
```

```
Missing values per column:
school        0
sex           0
age           0
address       0
famsize       0
Pstatus       0
Medu          0
Fedu          0
Mjob          0
Fjob          0
reason        0
guardian      0
traveltime    0
studytime     0
failures      0
schoolsup     0
famsup        0
paid          0
activities    0
nursery       0
higher        0
internet      0
romantic      0
famrel        0
freetime      0
goout         0
Dalc          0
Walc          0
health        0
```

```
absences      0
G1            0
G2            0
G3            0
dtype: int64
```

```
# Check duplicates
print("Duplicate rows:", df.duplicated().sum())
```

```
Duplicate rows: 0
```

```
# Convert binary categorical columns
binary_map = {"yes": 1, "no": 0}

binary_columns = [
    "schoolsup", "famsup", "paid", "activities",
    "nursery", "higher", "internet", "romantic"
]

for col in binary_columns:
    df[col] = df[col].map(binary_map)

df["sex"] = df["sex"].map({"F": 0, "M": 1})
df["address"] = df["address"].map({"R": 0, "U": 1})
df["famsize"] = df["famsize"].map({"LE3": 0, "GT3": 1})
df["Pstatus"] = df["Pstatus"].map({"A": 0, "T": 1})

print("Simple categorical columns converted.")
```

```
Simple categorical columns converted.
```

```
# One-hot encode multi-category columns
cleaned_df = pd.get_dummies(
    df,
    columns=["school", "Mjob", "Fjob", "reason", "guardian"],
    drop_first=True
)

for col in cleaned_df.columns:
    if cleaned_df[col].dtype == "bool":
        cleaned_df[col] = cleaned_df[col].astype(int)

print("Cleaned dataset shape:", cleaned_df.shape)
cleaned_df.head()
```

```
Cleaned dataset shape: (395, 42)
```

	sex	age	address	famsize	Pstatus	Medu	Fedu	traveltime	studytime	failures	...	Mjob_teacher	Fjob_health	Fjob_oth
0	0	18	1	1	0	4	4	2	2	0	...	0	0	
1	0	17	1	1	1	1	1	1	2	0	...	0	0	
2	0	15	1	0	1	1	1	1	2	3	...	0	0	
3	0	15	1	1	1	4	2	1	3	0	...	0	0	
4	0	16	1	1	1	3	3	1	2	0	...	0	0	

5 rows × 42 columns

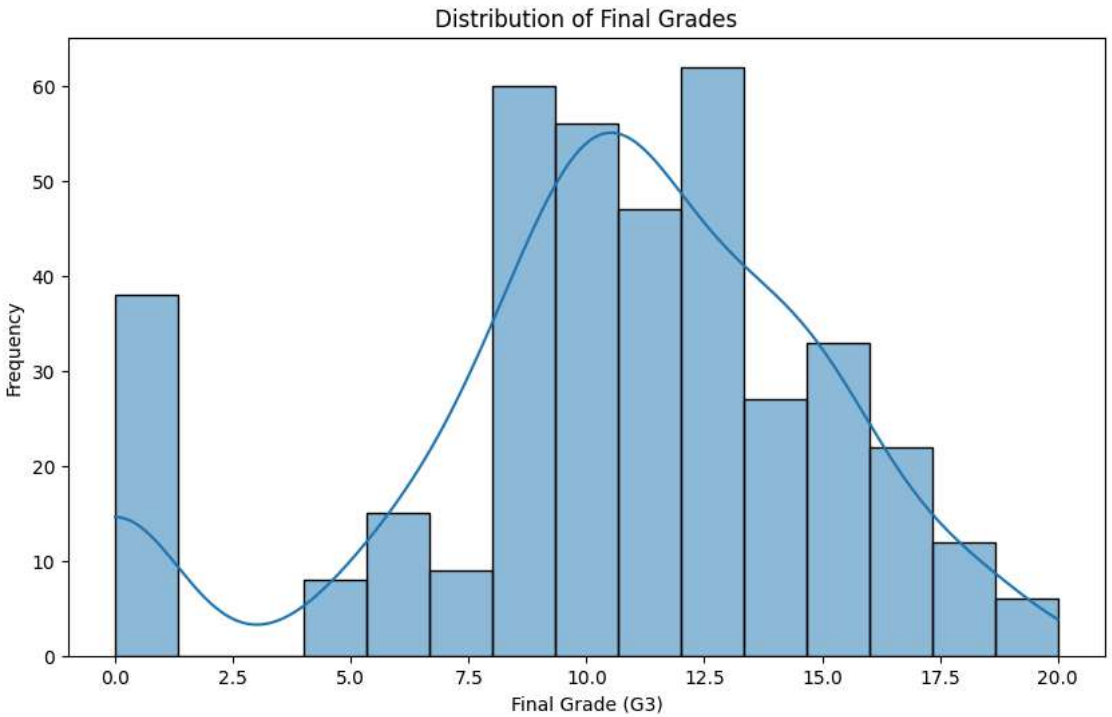
Exploratory Data Analysis

```
# 1. Basic statistics
numeric_cols = ["age", "Medu", "Fedu", "traveltime", "studytime", "failures",
                "famrel", "freetime", "goout", "Dalc", "Walc", "health",
                "absences", "G1", "G2", "G3"]

print("\nBasic statistics:")
print(cleaned_df[numeric_cols].describe())
```

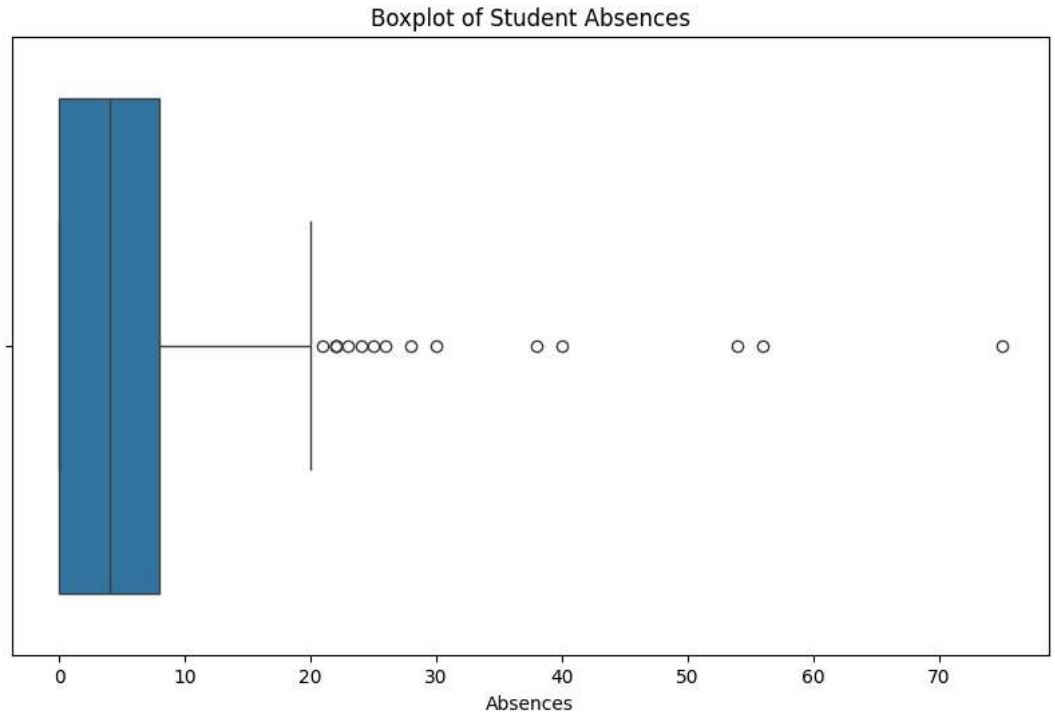
```
Basic statistics:
              age        Medu        Fedu  traveltime   studytime    failures  \
count  395.000000  395.000000  395.000000  395.000000  395.000000  395.000000
mean    16.696203    2.749367    2.521519    1.448101    2.035443    0.334177
std      1.276043    1.094735    1.088201    0.697505    0.839240    0.743651
min     15.000000    0.000000    0.000000    1.000000    1.000000    0.000000
25%     16.000000    2.000000    2.000000    1.000000    1.000000    0.000000
50%     17.000000    3.000000    2.000000    1.000000    2.000000    0.000000
75%     18.000000    4.000000    3.000000    2.000000    2.000000    0.000000
```

```
max     22.000000    4.000000    4.000000    4.000000    4.000000    3.000000

            famrel    freetime       goout        Dalc        Walc      health  \
count  395.000000  395.000000  395.000000  395.000000  395.000000  395.000000
mean     3.944304    3.235443    3.108861    1.481013    2.291139    3.554430
std      0.896659    0.998862    1.113278    0.890741    1.287897    1.390303
min      1.000000    1.000000    1.000000    1.000000    1.000000    1.000000
25%      4.000000    3.000000    2.000000    1.000000    1.000000    3.000000
50%      4.000000    3.000000    3.000000    1.000000    2.000000    4.000000
75%      5.000000    4.000000    4.000000    2.000000    3.000000    5.000000
max      5.000000    5.000000    5.000000    5.000000    5.000000    5.000000

          absences          G1          G2          G3
count  395.000000  395.000000  395.000000  395.000000
mean     5.708861   10.908861   10.713924   10.415190
std      8.003096    3.319195    3.761505    4.581443
min      0.000000    3.000000    0.000000    0.000000
25%      0.000000    8.000000    9.000000    8.000000
50%      4.000000   11.000000   11.000000   11.000000
75%      8.000000   13.000000   13.000000   14.000000
max     75.000000   19.000000   19.000000   20.000000
```

```
# 2. Histogram of final grade
plt.figure(figsize=(10, 6))
sns.histplot(data=cleaned_df, x="G3", bins=15, kde=True)
plt.title("Distribution of Final Grades")
plt.xlabel("Final Grade (G3)")
plt.ylabel("Frequency")
plt.show()
```



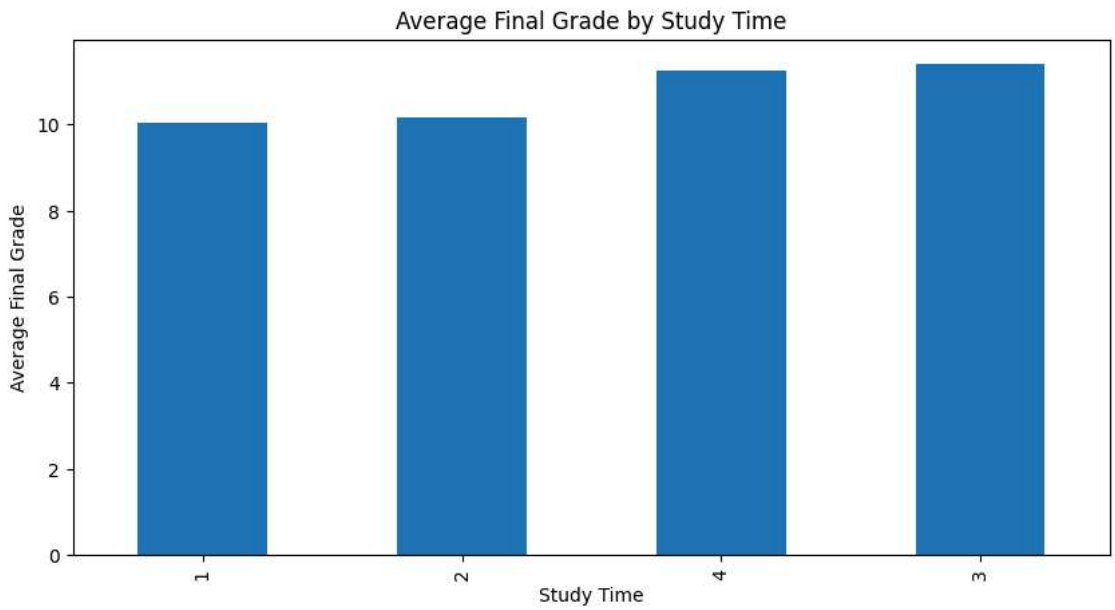

```
# 3. Boxplot of absences
plt.figure(figsize=(10, 6))
sns.boxplot(x=cleaned_df["absences"], orient="h")
plt.title("Boxplot of Student Absences")
plt.xlabel("Absences")
plt.show()
```




```
# 4. Average final grade by study time
avg_grade = cleaned_df.groupby("studytime")["G3"].mean().sort_values()
print(avg_grade)
```

```
studytime
1    10.047619
2    10.171717
4    11.259259
3    11.400000
Name: G3, dtype: float64
```

```
avg_grade.plot(kind="bar", figsize=(10, 5))
plt.title("Average Final Grade by Study Time")
plt.xlabel("Study Time")
plt.ylabel("Average Final Grade")
plt.show()
```



Principal Component Analysis (PCA)

```
# 5. Principal Component Analysis (PCA):
# Select relevant numeric variables for PCA

student_cols = [
```

```
    'age', 'Medu', 'Fedu', 'traveltime', 'studytime', 'failures',
    'famrel', 'freetime', 'goout', 'Dalc', 'Walc', 'health',
    'absences', 'G1', 'G2', 'G3'
]
```

```
# Create a subset with only student numeric data
student_df = cleaned_df[student_cols]
student_df.head()
```

	age	Medu	Fedu	traveltime	studytime	failures	famrel	freetime	goout	Dalc	Walc	health	absences	G1	G2	G3
0	18	4	4	2	2	0	4	3	4	1	1	3	6	5	6	6
1	17	1	1	1	2	0	5	3	3	1	1	3	4	5	5	6
2	15	1	1	1	2	3	4	3	2	2	3	3	10	7	8	10
3	15	4	2	1	3	0	3	2	2	1	1	5	2	15	14	15
4	16	3	3	1	2	0	4	3	2	1	2	5	4	6	10	10

Next steps: Generate code with `student_df` New interactive sheet

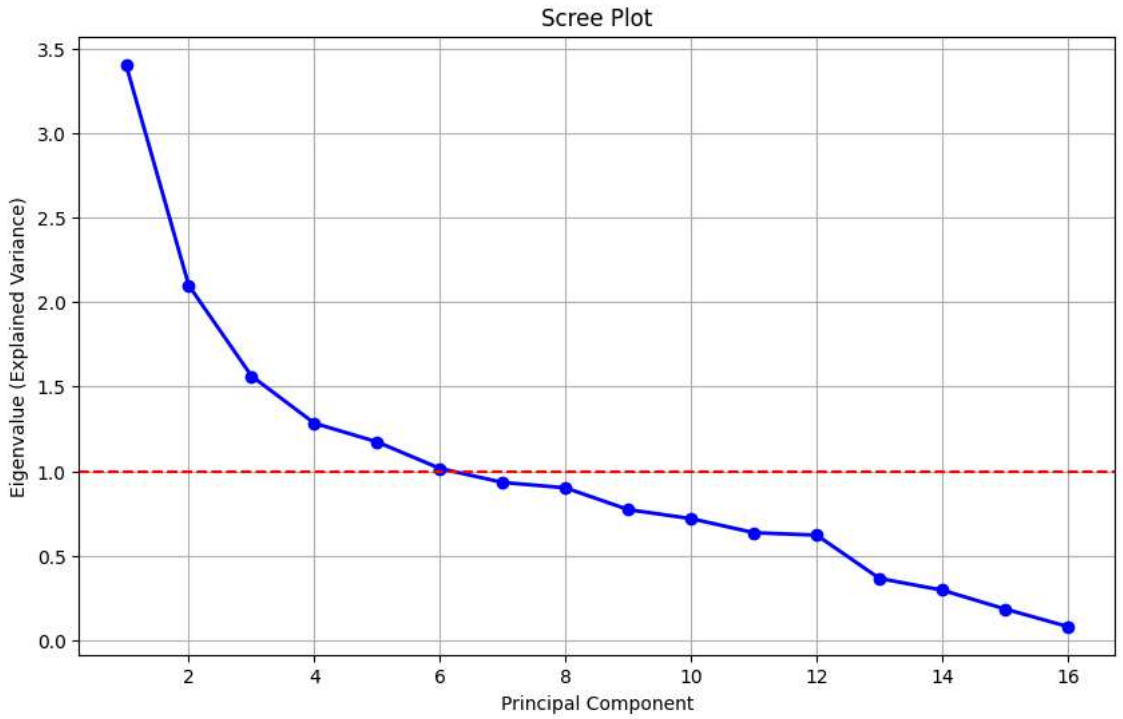
```
# Standardize the data (important for PCA)
scaler = StandardScaler()
student_scaled = scaler.fit_transform(student_df)
```

```
# Perform PCA
pca = PCA()
principal_components = pca.fit_transform(student_scaled)
```

```
# Eigenvalues (explained variance)
eigenvalues = pca.explained_variance_

print("Eigenvalues (Explained Variance):")
print(eigenvalues)
print("\n")

# Calculate total variation
total_variation = sum(eigenvalues)
print(f"Total Variation: {total_variation:.4f}")
```

```
Eigenvalues (Explained Variance):
[3.40422332 2.09764503 1.56165835 1.28293743 1.17259809 1.01530891
 0.93288936 0.90077492 0.77188583 0.71885914 0.63569    0.62093457
 0.365001   0.2957795  0.18414921 0.0802745 ]


Total Variation: 16.0406
```

```
# Scree plot
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))
plt.plot(range(1, len(eigenvalues)+1), eigenvalues, 'bo-', linewidth=2)
plt.title('Scree Plot')
plt.xlabel('Principal Component')
plt.ylabel('Eigenvalue (Explained Variance)')
plt.axhline(y=1, color='r', linestyle='--')  # Kaiser criterion line
plt.grid()
plt.show()
```




```
# Calculate proportion of variation for each component
proportions = eigenvalues / total_variation

print("\nProportion of Total Variation for Each Principal Component:")
for i, prop in enumerate(proportions):
    print(f"PC{i+1}: {prop:.4f} ({prop*100:.2f}%)")
```

```
Proportion of Total Variation for Each Principal Component:
PC1: 0.2122 (21.22%)
PC2: 0.1308 (13.08%)
PC3: 0.0974 (9.74%)
PC4: 0.0800 (8.00%)
PC5: 0.0731 (7.31%)
PC6: 0.0633 (6.33%)
PC7: 0.0582 (5.82%)
PC8: 0.0562 (5.62%)
PC9: 0.0481 (4.81%)
PC10: 0.0448 (4.48%)
PC11: 0.0396 (3.96%)
PC12: 0.0387 (3.87%)
PC13: 0.0228 (2.28%)
PC14: 0.0184 (1.84%)
PC15: 0.0115 (1.15%)
PC16: 0.0050 (0.50%)
```

```
# Cumulative proportion
cumulative_prop = proportions.cumsum()

print("\nCumulative Proportion of Total Variation:")
for i, cum_prop in enumerate(cumulative_prop):
    print(f"PC1 to PC{i+1}: {cum_prop:.4f} ({cum_prop*100:.2f}%)")
```

```
Cumulative Proportion of Total Variation:
PC1 to PC1: 0.2122 (21.22%)
PC1 to PC2: 0.3430 (34.30%)
PC1 to PC3: 0.4404 (44.04%)
PC1 to PC4: 0.5203 (52.03%)
PC1 to PC5: 0.5934 (59.34%)
PC1 to PC6: 0.6567 (65.67%)
PC1 to PC7: 0.7149 (71.49%)
PC1 to PC8: 0.7710 (77.10%)
PC1 to PC9: 0.8192 (81.92%)
PC1 to PC10: 0.8640 (86.40%)
PC1 to PC11: 0.9036 (90.36%)
PC1 to PC12: 0.9423 (94.23%)
PC1 to PC13: 0.9651 (96.51%)
PC1 to PC14: 0.9835 (98.35%)
PC1 to PC15: 0.9950 (99.50%)
PC1 to PC16: 1.0000 (100.00%)
```

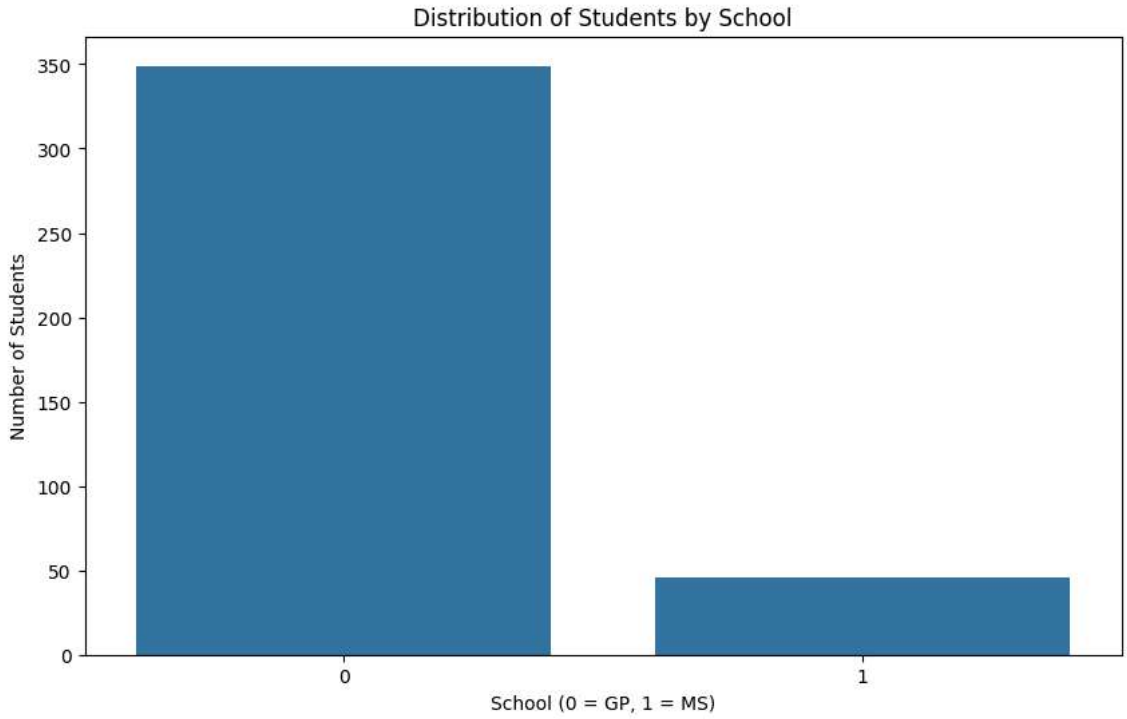
```python
# Eigenvectors (loadings)
eigenvectors = pca.components_

print("\nEigenvectors (Principal Components Loadings):")

for i, component in enumerate(eigenvectors):
    print(f"\nPrincipal Component {i+1}:")
    print(pd.Series(component, index=student_cols))
```

```
dtype: float64

Principal Component 14:
age           0.068443
Medu          0.236235
Fedu         -0.126625
traveltime    0.018030
studytime     0.084392
failures      0.009745
famrel        0.040069
freetime      0.130749
goout        -0.290756
Dalc         -0.559609
Walc          0.692831
health       -0.031551
absences     -0.062094
G1           -0.107656
G2           -0.013069
G3            0.053883
dtype: float64

Principal Component 15:
age          -0.114322
Medu          0.076799
Fedu         -0.122791
traveltime   -0.046259
studytime    -0.042282
failures      0.009969
famrel        0.041164
freetime     -0.030957
goout        -0.017255
Dalc         -0.023633
Walc          0.107991
health       -0.013984
absences      0.043383
G1            0.777516
G2           -0.218885
G3           -0.540590
dtype: float64

Principal Component 16:
age          -0.005754
Medu          0.002860
Fedu         -0.002345
traveltime    0.037801
studytime    -0.011696
failures     -0.017709
famrel        0.050060
freetime      0.015658
goout         0.011007
Dalc         -0.025837
Walc          0.012766
health        0.022857
absences      0.048754
G1           -0.200121
G2            0.774843
G3           -0.592488
dtype: float64
```

```python
# School-wise data balance
print(cleaned_df['school_MS'].value_counts())
```

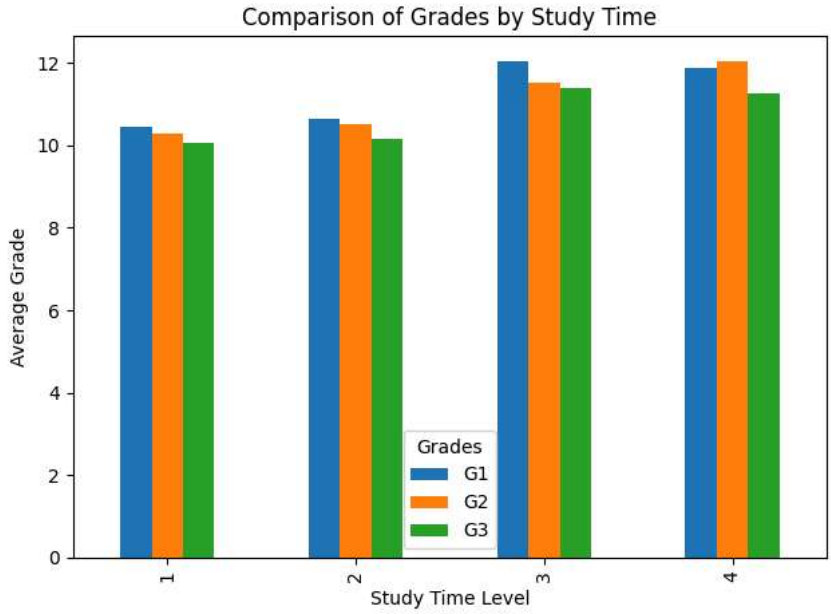
```
school_MS
0    349
1     46
Name: count, dtype: int64
```

```python
# Count plot for school distribution
plt.figure(figsize=(10,6))
sns.countplot(x='school_MS', data=cleaned_df)
plt.xlabel("School (0 = GP, 1 = MS)")
plt.ylabel("Number of Students")
plt.title("Distribution of Students by School")
plt.show()
```



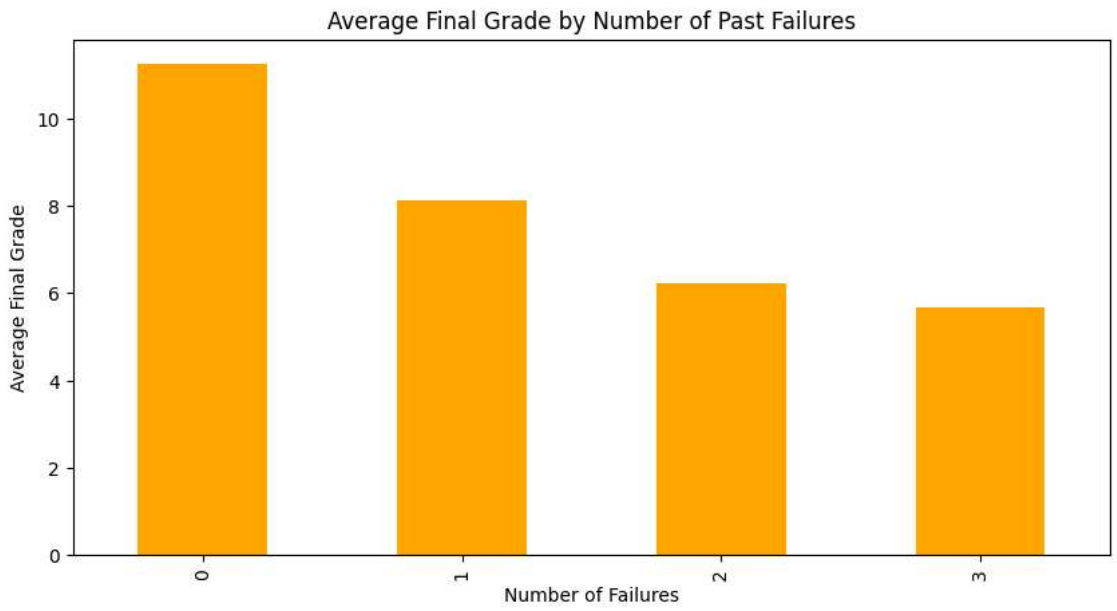

```
data = cleaned_df.copy()

study_stats = data.groupby('studytime')[['G1','G2','G3']].mean()
print(study_stats)

plt.figure(figsize=(12,6))
study_stats.plot(kind='bar')
plt.title("Comparison of Grades by Study Time")
plt.xlabel("Study Time Level")
plt.ylabel("Average Grade")
plt.legend(title="Grades")
plt.tight_layout()
plt.show()
```

```
                 G1         G2         G3
studytime
1          10.438095  10.276190  10.047619
2          10.651515  10.505051  10.171717
3          12.046154  11.507692  11.400000
4          11.888889  12.037037  11.259259
<Figure size 1200x600 with 0 Axes>
```

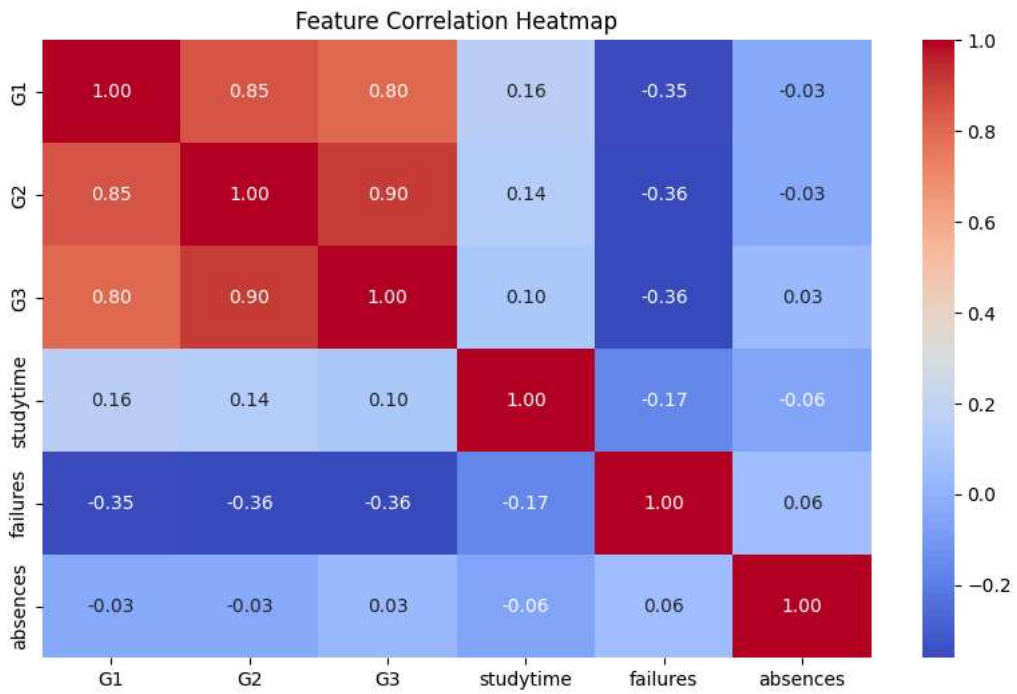



```
fail_stats = data.groupby('failures')['G3'].mean()

plt.figure(figsize=(10,5))
fail_stats.plot(kind='bar', color='orange')
```

```
plt.title("Average Final Grade by Number of Past Failures")
plt.xlabel("Number of Failures")
plt.ylabel("Average Final Grade")
plt.show()
```

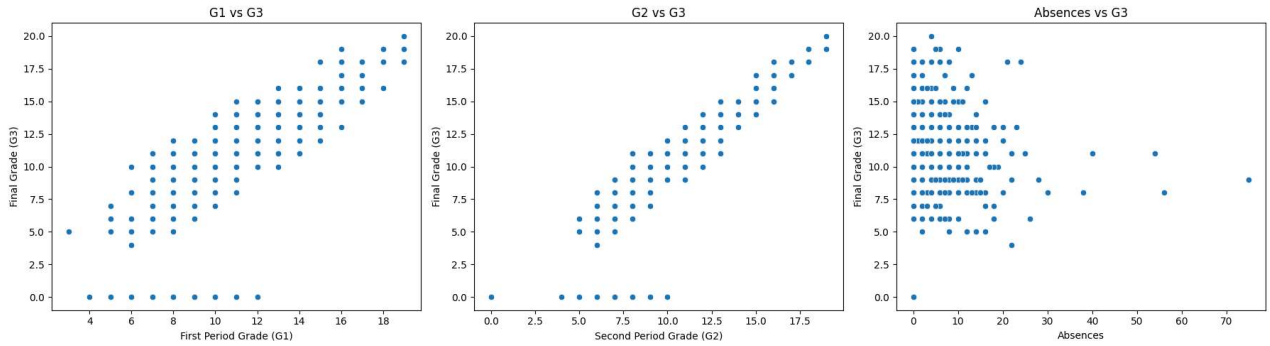



```
# Correlation heatmap
corr_cols = ["G1", "G2", "G3", "studytime", "failures", "absences"]
plt.figure(figsize=(10, 6))
sns.heatmap(cleaned_df[corr_cols].corr(), annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Feature Correlation Heatmap")
plt.show()
```




```
# Set up the figure
plt.figure(figsize=(18, 5))

# Plot 1: G1 vs G3
plt.subplot(1, 3, 1)
sns.scatterplot(x="G1", y="G3", data=cleaned_df)
plt.title("G1 vs G3")
plt.xlabel("First Period Grade (G1)")
plt.ylabel("Final Grade (G3)")

# Plot 2: G2 vs G3
plt.subplot(1, 3, 2)
sns.scatterplot(x="G2", y="G3", data=cleaned_df)
plt.title("G2 vs G3")
plt.xlabel("Second Period Grade (G2)")
```

```
plt.ylabel("Final Grade (G3)")

# Plot 3: Absences vs G3
plt.subplot(1, 3, 3)
sns.scatterplot(x="absences", y="G3", data=cleaned_df)
plt.title("Absences vs G3")
plt.xlabel("Absences")
plt.ylabel("Final Grade (G3)")

plt.tight_layout()
plt.show()
```



ML / AI Methods

```
# Select features and target
X = cleaned_df[["studytime", "failures", "absences", "schoolsup",
                "famsup", "higher", "internet", "G1", "G2"]]
y = cleaned_df["G3"]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=88
)
```

Linear Regression

```
# Linear Regression
lr_model = LinearRegression()
lr_model.fit(X_train, y_train)
lr_pred = lr_model.predict(X_test)
```

```
print("\nLinear Regression Performance:")
print("MAE:", mean_absolute_error(y_test, lr_pred))
print("MSE:", mean_squared_error(y_test, lr_pred))
print("R2 Score:", r2_score(y_test, lr_pred))
```

```
Linear Regression Performance:
MAE: 1.19117211787797
MSE: 3.3251279737959267
R2 Score: 0.8556105349176844
```

Random Forest Regressor

```
# Random Forest Regressor
rf_model = RandomForestRegressor(n_estimators=100, random_state=88)
rf_model.fit(X_train, y_train)
rf_pred = rf_model.predict(X_test)
```

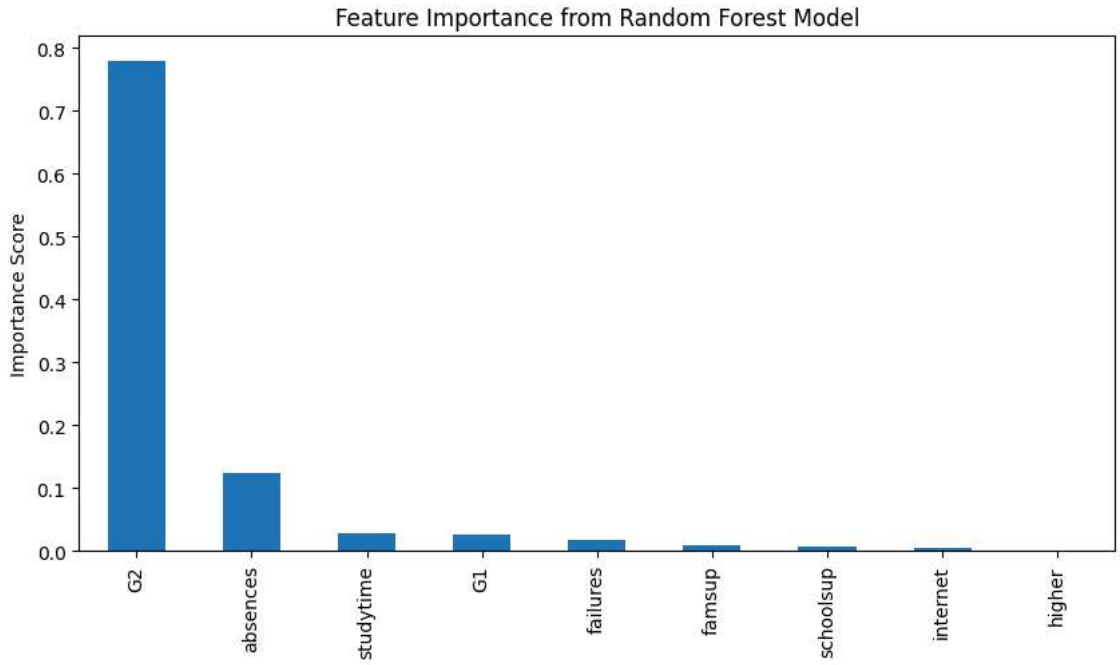
```
print("\nRandom Forest Performance:")

rf_mae = mean_absolute_error(y_test, rf_pred)
rf_mse = mean_squared_error(y_test, rf_pred)
```

```
rf_r2 = r2_score(y_test, rf_pred)

print("MAE:", rf_mae)
print("MSE:", rf_mse)
print("R2 Score:", rf_r2)
```

```
Random Forest Performance:
MAE: 0.9310074029611844
MSE: 2.2240278919925114
R2 Score: 0.9034244094712677
```

```
# Feature Importance
importance = pd.Series(
    rf_model.feature_importances_,
    index=X.columns
).sort_values(ascending=False)

print("\nFeature Importance:")
print(importance)

plt.figure(figsize=(10,5))
importance.plot(kind='bar')
plt.title("Feature Importance from Random Forest Model")
plt.ylabel("Importance Score")
plt.show()
```

```
Feature Importance:
G2           0.780183
absences     0.124143
studytime    0.028493
G1           0.026518
failures     0.017630
famsup       0.010210
schoolsup    0.006497
internet     0.005229
higher       0.001098
dtype: float64
```



Neural Network Model - Student Performance Prediction

```
# Neural Network Model
nn_model = Sequential([
    Dense(128, activation='relu', input_shape=(X_train.shape[1],)),
    Dropout(0.2),
    Dense(64, activation='relu'),
    Dropout(0.2),
    Dense(32, activation='relu'),
    Dense(1)   # Output layer for regression
])

# Compile the model
nn_model.compile(optimizer='adam', loss='mse', metrics=['mae'])
```

```
# Train the model
history = nn_model.fit(
    X_train, y_train,
    epochs=100,
    batch_size=16,
    validation_data=(X_test, y_test),
    verbose=1
)
```

```
Epoch 36/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 4.3720 - mae: 1.5521 - val_loss: 2.8870 - val_mae: 1.1991
Epoch 37/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 4.2681 - mae: 1.4459 - val_loss: 3.1107 - val_mae: 1.2846
Epoch 38/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 3.5980 - mae: 1.4168 - val_loss: 4.9505 - val_mae: 1.8504
Epoch 39/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 4.2295 - mae: 1.4554 - val_loss: 4.1455 - val_mae: 1.6222
Epoch 40/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 3.9035 - mae: 1.3964 - val_loss: 6.0171 - val_mae: 2.1073
Epoch 41/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 3.8410 - mae: 1.4050 - val_loss: 4.1555 - val_mae: 1.6348
Epoch 42/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 3.8539 - mae: 1.4420 - val_loss: 3.8926 - val_mae: 1.5458
Epoch 43/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - loss: 3.9392 - mae: 1.4424 - val_loss: 3.3866 - val_mae: 1.3962
Epoch 44/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 3.4311 - mae: 1.3511 - val_loss: 5.3852 - val_mae: 1.9626
Epoch 45/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 4.1489 - mae: 1.4851 - val_loss: 5.8452 - val_mae: 2.0647
Epoch 46/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 3.9576 - mae: 1.4353 - val_loss: 5.0409 - val_mae: 1.8680
Epoch 47/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 3.8322 - mae: 1.3740 - val_loss: 3.0997 - val_mae: 1.3056
Epoch 48/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 3.7173 - mae: 1.3710 - val_loss: 4.1507 - val_mae: 1.6211
Epoch 49/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 3.6818 - mae: 1.3836 - val_loss: 3.6507 - val_mae: 1.4698
Epoch 50/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 4.0749 - mae: 1.4600 - val_loss: 3.9532 - val_mae: 1.5583
Epoch 51/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 3.7425 - mae: 1.4245 - val_loss: 4.4277 - val_mae: 1.7077
Epoch 52/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 4.0238 - mae: 1.4632 - val_loss: 3.0422 - val_mae: 1.2850
Epoch 53/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 3.7581 - mae: 1.4595 - val_loss: 3.8067 - val_mae: 1.5333
Epoch 54/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 3.5091 - mae: 1.3689 - val_loss: 3.7766 - val_mae: 1.5266
Epoch 55/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 3.7660 - mae: 1.4101 - val_loss: 6.5589 - val_mae: 2.2161
Epoch 56/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 3.6877 - mae: 1.3900 - val_loss: 6.8182 - val_mae: 2.2688
Epoch 57/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 3.6681 - mae: 1.3817 - val_loss: 6.2311 - val_mae: 2.1523
Epoch 58/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 3.7432 - mae: 1.4098 - val_loss: 3.9907 - val_mae: 1.5821
Epoch 59/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 4.1181 - mae: 1.4818 - val_loss: 4.6341 - val_mae: 1.7764
Epoch 60/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 3.8754 - mae: 1.4154 - val_loss: 4.1155 - val_mae: 1.6206
Epoch 61/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 3.8611 - mae: 1.4318 - val_loss: 4.6960 - val_mae: 1.7884
Epoch 62/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 3.1491 - mae: 1.3255 - val_loss: 4.5976 - val_mae: 1.7591
Epoch 63/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 3.6175 - mae: 1.3894 - val_loss: 3.1930 - val_mae: 1.3303
Epoch 64/100
```

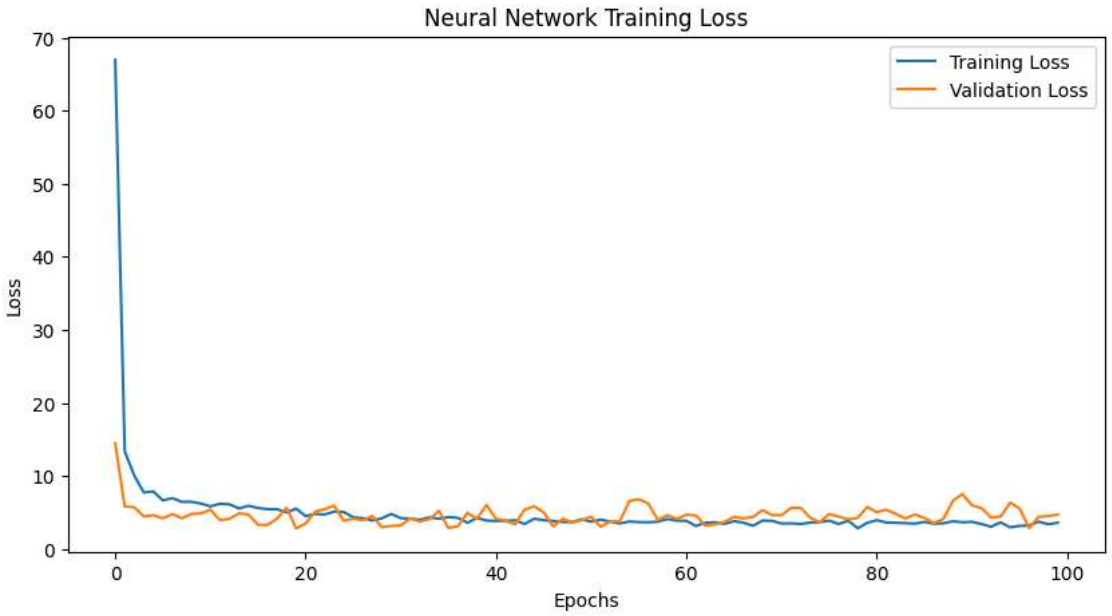
Evaluate the Neural Network

```
# Predict on test data
nn_pred = nn_model.predict(X_test).flatten()

# Evaluation metrics
nn_mae = mean_absolute_error(y_test, nn_pred)
nn_mse = mean_squared_error(y_test, nn_pred)
nn_r2 = r2_score(y_test, nn_pred)

print("\nNeural Network Performance:")
print("MAE:", nn_mae)
print("MSE:", nn_mse)
print("R2 Score:", nn_r2)
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step

Neural Network Performance:
MAE: 1.7802464962005615
```

```
MSE: 4.705598831176758
R2 Score: 0.7956653833389282
```

```
#Training Graph
plt.figure(figsize=(10, 5))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title("Neural Network Training Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()
```



Start coding or generate with AI.